

PRODUCT REQUIREMENTS DOCUMENT

SenSai

CEO Heads-Up Display & Knowledge Wiki

Migrating the gstack agentic framework into an executive operating system

Document type	Product Requirements Document (PRD) — migration + net-new capability
Base system	gstack (garrytan/gstack) — Garry Tan's opinionated Claude Code framework
Fork under migration	metacogna-lab/sensai-stack (SenSai)
Companion pattern	Karpathy LLM Wiki (raw → wiki → schema, compile-once knowledge)
Target persona	CEO / founder-operator and their executive assistant
Build surface	Claude Code · Claude Cowork · agent skills (SKILL.md)
Status	Draft v1.0 — for engineering + design review
Date	1 July 2026

Prepared by **Agentic Engineering / AI Engineering**

Contents

0. Sourcing & scope note

This PRD was assembled from primary sources. The **gstack** base architecture is drawn from the upstream repository *garrytan/gstack* (ARCHITECTURE.md, docs/skills.md), and the **Karpathy LLM Wiki** pattern from the canonical gist *karpathy/442a6bf555914893e9891c11519de94f* plus its production-hardening discussion.

Note on the fork repository

At the time of writing, github.com/metacogna-lab/sensai-stack.git is not publicly resolvable (private or not yet indexed). This PRD therefore treats **SenSai** as a fork of the canonical upstream *gstack* and specifies the migration against that known-good base. Where the private fork already diverges, engineering should reconcile Section 8 (Migration) against the fork's actual HEAD before executing the rename.

1. Executive summary

gstack is a set of 23 opinionated Claude Code skills that let a single operator act as **CEO, Designer, Engineering Manager, Release Manager, Doc Engineer, and QA**. Its hard engineering core is a long-lived Chromium daemon (sub-second, stateful browser control) with everything else expressed as Markdown skills, a self-generating SKILL.md template system, and an optional gbrain memory layer.

The Karpathy LLM Wiki is a complementary pattern: instead of re-deriving knowledge from raw documents on every query (RAG), an LLM **compiles** durable, interlinked Markdown pages once and keeps them current. Raw sources are immutable; the wiki is the compounding artifact; a schema file governs how the agent ingests, queries, and lints.

SenSai fuses the two. It keeps *gstack*'s role-based execution engine and browser, retargets the operator lens from "solo founder shipping code" to "**CEO running a company**," and adds a persistent, LLM-maintained knowledge substrate for three new first-class objects: a **CEO Heads-Up Display (HUD)**, **Client Notes**, and an **Executive Assistant** agent. The whole codebase is renamed from *gstack* to *sensai*.

The one-paragraph pitch

SenSai

A CEO opens one interface each morning. SenSai has already read the overnight signal — email, calendar, docs, Slack, client activity — compiled it into a maintained wiki, flagged contradictions and stale commitments, and rendered a single **Heads-Up Display**: what needs a decision today, what each client is waiting on, what slipped, and what the assistant already handled. The CEO curates and decides; SenSai does the bookkeeping.

What changes vs. base *gstack*

Dimension	<i>gstack</i> (base)	SenSai (target)
Primary user	Solo founder / engineer	CEO / founder-operator + executive assistant
Core output	Shipped code, PRs, QA	Decisions, client outcomes, delegated actions
Knowledge model	gbrain memory + code context	Karpathy LLM Wiki (raw→wiki→schema) + gbrain
Headline surface	Slash commands in terminal	CEO Heads-Up Display (HUD) briefing

Dimension	gstack (base)	SenSai (target)
New objects	—	Client Notes · Exec Assistant · HUD
Namespace	gstack / \$B / ~/.gstack	sensai / \$S / ~/.sensai

2. Goals & non-goals

2.1 Product goals

1. **Compile a live executive picture.** Maintain an always-current wiki of the company, its clients, commitments, and decisions — never re-derived from scratch.
2. **Render a single daily HUD.** One briefing surface answering: what decides today, what's at risk, what each client needs, what the assistant handled.
3. **Make client relationships a first-class object.** Structured, append-only Client Notes with commitments, sentiment, open threads, and next actions.
4. **Delegate the operator's overhead.** An Executive Assistant agent that drafts, schedules, chases, and prepares — with human-in-the-loop gates on anything it sends.
5. **Preserve gstack's execution muscle.** Keep the browser daemon, ref system, and role skills; retarget, don't rebuild.
6. **Rename cleanly.** Every gstack reference becomes sensai, with zero broken paths, binaries, or skill invocations.

2.2 Non-goals (v1)

- Not a CRM replacement. SenSai reads from and links to CRMs; it does not become the system of record for deals.
- Not multi-tenant. Like gstack, one workspace = one principal (the CEO). The assistant is an agent, not a second seat.
- Not autonomous outbound. The assistant never sends external communications without an explicit approval gate in v1.
- Not embedding-RAG infrastructure. Follow the wiki's index-first approach; add hybrid search only when scale demands it.
- Not a mobile-native app. v1 targets Claude Code / Cowork surfaces; a HUD web view is a stretch goal.

2.3 Success metrics

Metric	Definition	v1 target
Morning-to-decision time	Time from opening HUD to first logged decision	< 10 min
Commitment recall	% of client commitments surfaced before due date	> 95%
Contradiction catch rate	Stale/conflicting facts flagged at ingest	> 90% (hard conflicts block)
Assistant deflection	Routine actions drafted without CEO initiation	≥ 40% of weekly ops volume
Wiki freshness	Overview lag behind newest ingest	≤ 1 ingest cycle
Rename integrity	Broken gstack refs after migration	0

3. Background: the two systems we are fusing

3.1 gstack — the execution engine we inherit

gstack's design rests on a small number of load-bearing ideas that SenSai preserves verbatim:

- **Long-lived browser daemon.** A Chromium process behind a localhost HTTP server (Bun.serve). First call ~3s; every call after ~100–200ms. Cookies, tabs, and logins persist. Auto-starts on use, auto-shuts after 30 min idle.
- **The ref system.** Elements are addressed as @e1/@c1 via Playwright Locators over the accessibility tree — no CSS/XPath, no DOM mutation, robust to CSP and framework hydration. Refs clear on navigation and fail loudly when stale.
- **Everything else is Markdown.** Skills are SKILL.md files generated from a template system: human-written workflows + placeholders filled from source at build time, so docs never drift from code.
- **A preamble on every skill.** Update check, session tracking ("ELI16 mode" when juggling 3+ windows), operational self-improvement logging, a consistent AskUserQuestion format, and a Search-Before-Building discipline.
- **Role skills.** /plan-ceo-review, /plan-eng-review, /design-review, /qa, /ship, /review, /codex, /cso and more — each a role the operator steps into.
- **gbrain memory (optional).** A self-wiring knowledge graph with hybrid vector+graph retrieval; skills load brain context in and save results back.

Design tension to respect

gstack's ARCHITECTURE.md explicitly rejects MCP and multi-user for the browser core (token overhead, complexity). SenSai's new surfaces (HUD, wiki, client sync) *may* use MCP connectors for data-in, but the browser-driving path stays plain-HTTP and single-principal. Do not entangle the two.

3.2 Karpathy LLM Wiki — the knowledge substrate we add

The wiki pattern replaces "retrieve-and-forget" with "compile-and-compound." Three layers:

Layer	Owner	Role in SenSai
raw/	Human / connectors (immutable)	Source of truth: emails, transcripts, docs, client threads, CRM exports
wiki/	The LLM (writes everything)	Compiled pages: clients, people, decisions, commitments, the overview & HUD feed
schema (SENSAI.md)	Human + LLM co-evolve	How to ingest, query, lint; page types; conventions; approval gates

Three operations run the loop: **Ingest** (a source touches 10–15 pages), **Query** (index-first, answers can be filed back as pages), and **Lint** (contradictions, stale claims, orphans, coverage gaps). Two navigation files — index.md (content catalog) and log.md (append-only timeline) — keep it operable without embedding infrastructure.

Production lessons folded in (from the gist's field reports)

- **Contradiction severity gate.** Classify each conflict soft / scope-mismatch / hard. Hard conflicts block the ingest commit until a human resolves them.

- **Deterministic commit gate.** A cheap os.walk + grep for "Status: Unresolved" — no LLM tokens — scales to any size.
- **Scoped linting.** Only re-lint changed pages + graph neighbors; periodic full sweeps as cleanup, not primary defense.
- **Page lifecycle status.** Each page carries a confidence/verification state so new sources re-examine, not rebuild.

4. Personas & top jobs-to-be-done

Primary: the CEO / founder-operator

- "Tell me what actually needs me today — not everything, the decisions."
- "What is each important client waiting on, and did we do what we said?"
- "What slipped, what's at risk, and what did my assistant already handle?"
- "Keep the context so I never re-explain the same situation twice."

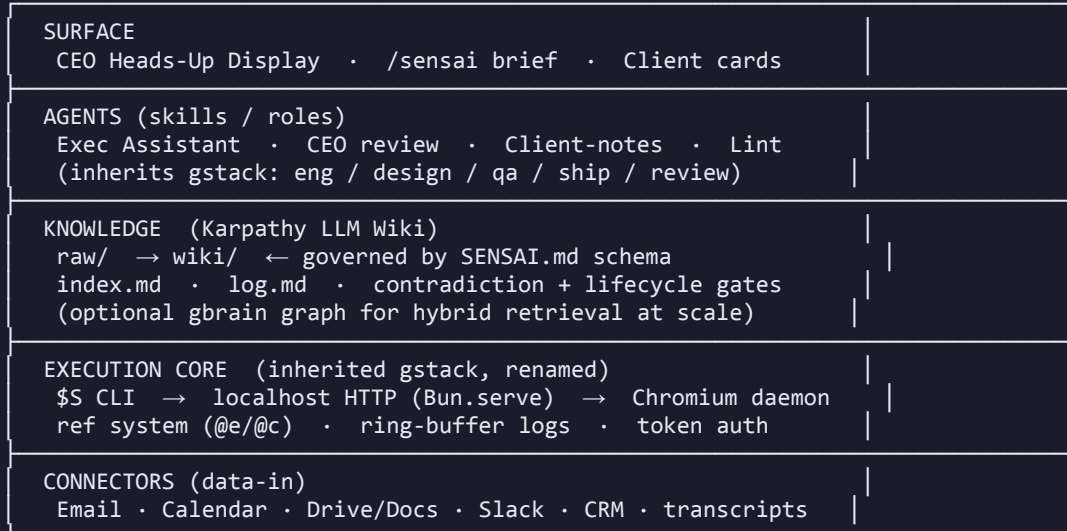
Secondary: the executive assistant (human) + the Exec Assistant agent

- "Draft the follow-ups, prep the meetings, chase the open threads — I'll approve."
- "Turn a messy call transcript into client notes and next actions automatically."
- "Never let a commitment fall through; escalate the ones that need the principal."

5. Target system architecture

SenSai is gstack's daemon + skills core, wrapped by a wiki-backed knowledge layer, topped by the HUD. Data flows in from connectors, is compiled into the wiki, and is rendered as the HUD; actions flow back out through gated skills.

5.1 Layer diagram



5.2 Wiki directory layout

```
sensai/
├── SENSAI.md           # schema (was CLAUDE.md) – the operating manual
├── raw/                # immutable sources (agent reads, never writes)
│   ├── email/ calendar/ transcripts/ crm/ docs/
│   └── clients/<client-slug>/ # per-client raw drops
├── wiki/              # LLM-owned compiled knowledge
│   ├── index.md       # catalog: every page, one-line summary
│   ├── log.md         # append-only: ## [YYYY-MM-DD] ingest|query|lint
│   ├── overview.md    # evolving company synthesis
│   ├── hud/
│   │   └── today.md   # rendered daily HUD feed
│   ├── clients/<client>.md # CLIENT NOTES (see §7)
│   ├── people/<person>.md # stakeholders, contacts
│   ├── decisions/<slug>.md # decision log w/ rationale + status
│   ├── commitments/   # promises made, owner, due, status
│   └── concepts/<topic>.md # products, markets, initiatives
└── .sensai/           # daemon state, sessions, security, models
```

5.3 Page frontmatter contract

Every wiki page carries YAML frontmatter so lint and the HUD can query deterministically (mirrors the field-tested linter schema).

```
---
type: client           # client | person | decision | commitment | concept | hud
title: Acme Corp
```

```
status: active          # active | stale | archived
confidence: high        # high | medium | low (lifecycle signal)
contradiction: none     # none | soft | scope | hard
owner: ceo
due:                    # for commitments
sources: [raw/clients/acme/2026-06-30-call.md]
tags: [enterprise, renewal]
updated: 2026-07-01
---
```

6. Feature: CEO Heads-Up Display (HUD)

The HUD is a **rendered query**, not a database. Each morning (or on demand via `/sensai brief`), the agent reads the index, pulls the relevant wiki pages, and compiles `wiki/hud/today.md` — which is itself filed back into the wiki so the daily picture compounds and becomes auditable over time.

6.1 HUD sections (v1)

Section	Answers	Compiled from
Decide Today	What needs the CEO's judgment now	decisions/ (open) + commitments due + flagged conflicts
At Risk	What slipped or is trending wrong	commitments (overdue) + sentiment drops in client notes
Client Pulse	Per-client: waiting-on, last touch, next action	clients/*.md
Handled	What the Exec Assistant did / drafted	log.md (assistant entries, last 24h)
Signal In	New sources ingested overnight	log.md ingest entries + overview delta

6.2 HUD generation flow

1. Preamble runs (update check, session tracking, self-improvement log) — inherited from gstack.
2. Read `index.md`; select pages by frontmatter (`status=active`, `due` within horizon, `contradiction≠none`).
3. Rank: decisions and hard-conflicts first, then overdue commitments, then client next-actions.
4. Render `today.md` with citations back to source pages; write it to `wiki/hud/` and append a log entry.
5. Offer 2–4 next actions via the standard `AskUserQuestion` format (each with a recommendation).

Acceptance criteria — HUD

- Running `/sensai brief` with an empty wiki returns a graceful "no signal yet" state, not an error.
- Every HUD line links to at least one wiki page; no un-sourced claims.
- Hard contradictions appear in Decide Today and block silent auto-actions.
- `today.md` is idempotent per day (re-running updates in place, one log entry per generation).

7. Feature: Client Notes

Client Notes make the relationship a first-class, LLM-maintained wiki object. One page per client, **append-only in spirit**: the agent adds and revises structured sections as new sources arrive, but never silently discards prior commitments — superseded items are marked, not deleted.

7.1 Client page structure

```
# Acme Corp                                [type: client | status: active]

## Snapshot
One-paragraph current state. Auto-revised each ingest.

## Waiting On (us → them / them → us)
- [ ] Send revised SOW (owner: ceo, due: 2026-07-03) ← commitment
- [ ] Their legal review of MSA (them, due: 2026-07-05)

## Commitments Ledger
| Promise | Made | Owner | Due | Status |
| ...     | ...  | ...   | ... | open/kept/slipped |

## Sentiment & Signals
Trend line from call transcripts + email tone. Flag drops.

## Open Threads
Live topics, each linked to source + relevant decision pages.

## People
[[people/jane-doe]] (champion), [[people/john-roe]] (economic buyer)

## History
Append-only log of touches (calls, emails, meetings).
```

7.2 Ingest → Client Notes flow

1. A source lands in raw/clients/<client>/ (transcript, email thread, CRM export).
2. Agent reads it, discusses key takeaways, and updates the client page: snapshot, waiting-on, ledger, sentiment.
3. New promises become commitment entries (own frontmatter + due date) and cross-link into commitments/.
4. Contradictions (e.g., a new price vs. an old quote) are severity-classified; hard conflicts block and notify.
5. index.md, overview.md, and any touched people/ pages are updated; a log entry is appended.

Privacy & trust boundary

Client Notes contain sensitive relationship data. Per the gstack security model, raw client material stays local; the daemon binds to 127.0.0.1; anything routed to a cloud model for synthesis is an explicit, logged boundary — not the default for raw private data. Redaction tags in frontmatter mark fields that must never leave the machine.

Acceptance criteria — Client Notes

- Every commitment has an owner, a due date, and a status; overdue ones surface in the HUD At-Risk section.

- Superseded commitments are struck and annotated with the superseding source, never deleted.
- A new client can be bootstrapped from a single transcript in one ingest.
- Sentiment drops of a defined threshold raise a flag on the client card.

8. Feature: Executive Assistant agent

The Exec Assistant is a gstack-style role skill (`/sensai assist`) that turns the compiled wiki into drafted, gated action. It is the operator's overhead-absorber: it prepares and proposes; the CEO approves.

8.1 Capabilities (v1)

Capability	What it does	Gate
Follow-up drafting	Drafts client emails from open Waiting-On items	CEO approves before send
Meeting prep	Builds a one-pager from the client page + last touches	None (read-only output)
Thread chasing	Lists stale open threads, drafts nudges	CEO approves before send
Transcript intake	Turns a call recording/transcript into client notes + actions	Auto-ingest; hard-conflict block
Calendar triage	Proposes reschedules/holds against priorities	CEO approves before write
Daily digest	Compiles the Handled section of the HUD	None (read-only)

8.2 The approval gate (non-negotiable in v1)

No external communication or calendar mutation happens without an explicit CEO approval step, surfaced in the standard AskUserQuestion format (context, question, *RECOMMENDATION: choose X because __*, lettered options). Every gated action and its resolution is written to log.md for auditability.

Reuse, don't reinvent

The assistant's outbound drafting can reuse gstack's existing message-composition patterns and the browser daemon for any web action (e.g., posting into a portal). Prompt-injection defenses (L1–L6, canary token) already guard the sidebar agent path and MUST remain enabled when the assistant reads hostile web content.

Acceptance criteria — Exec Assistant

- Zero external sends occur without a logged approval in v1.
- A transcript dropped in raw/ produces client-note updates + a draft follow-up within one ingest.
- Every assistant action appears in the HUD Handled section next morning.
- Injection-classifier BLOCK ends the action and logs layer scores; no partial send.

9. Migration: iterating gstack → SenSai

This section is executable. It assumes a clone of the fork (or upstream gstack) and Claude Code driving the edits. Work in a branch; the rename is mechanical but load-bearing, so it is gated behind the existing test tiers.

9.1 Rename map — every "gstack" becomes "sensai"

Class	gstack (before)	sensai (after)
Product name	gstack / GStack	sensai / SenSai
CLI binary + var	\$B (browse) / gstack	\$S / sensai
State dir	~/.gstack, ~/.gstack-dev	~/.sensai, ~/.sensai-dev
State file	.gstack/browse.json	.sensai/browse.json
Schema file	CLAUDE.md (gstack sections)	SENSAI.md
Env knobs	GSTACK_SECURITY_OFF, GSTACK_SECURITY_ENSEMBLE	SENSAI_SECURITY_OFF, SENSAI_SECURITY_ENSEMBLE
Update/util bins	gstack-update-check, gstack-security-dashboard	sensai-update-check, sensai-security-dashboard
Sessions/security	~/.gstack/sessions, ~/.gstack/security	~/.sensai/sessions, ~/.sensai/security
Cookie SSE cookie	gstack_sse	sensai_sse
Skill namespace	/browse, /ship, /qa ...	/sensai brief, /sensai assist, + retained roles

9.2 Mechanical rename procedure

Order matters

Rename **identifiers and paths first**, regenerate the SKILL.md docs **second** (they are generated from source — never hand-edit generated sections), and run the test tiers **last**. Because gstack's docs are generated from code metadata, a source-level rename propagates to docs on the next gen step.

- 1. Branch.** `git checkout -b rename/gstack-to-sensai`
- 2. Inventory.** Enumerate occurrences before editing:

```
# case-sensitive, whole-word-ish inventory
grep -rIn --exclude-dir=node_modules --exclude-dir=.git \
  -e 'gstack' -e 'GStack' -e 'GSTACK' -e '\$B\b' . | tee rename-inventory.txt
wc -l rename-inventory.txt
```

- 3. Guarded replace.** Replace by class, preserving case. Do the three case variants separately so GStack→SenSai and GSTACK→SENSAI stay correct:

```
# lowercase identifiers, paths, package name
grep -rIl --exclude-dir=node_modules --exclude-dir=.git 'gstack' . \
  | xargs sed -i 's/gstack/sensai/g'
# TitleCase product name
grep -rIl --exclude-dir=node_modules --exclude-dir=.git 'GStack' . \
  | xargs sed -i 's/GStack/SenSai/g'
# SCREAMING env knobs
```

```
grep -rIl --exclude-dir=node_modules --exclude-dir=.git 'GSTACK' . \
| xargs sed -i 's/GSTACK/SENSAI/g'
```

4. **Rename the CLI symbol.** Update the browse binary variable \$B → \$S in resolvers and every SKILL.md.tmpl; do not touch generated SKILL.md by hand.
5. **Move state dirs.** Rename ~/.gstack → ~/.sensai references and the .gstack/ workspace dir → .sensai/. Add a one-time shim that migrates an existing ~/.gstack on first run.
6. **Rename schema.** CLAUDE.md → SENSAI.md; port gstack sections and add the wiki schema (§9.4).
7. **Regenerate docs.** Run the skill-doc generator so {{PREAMBLE}}, {{COMMAND_REFERENCE}} etc. re-emit with sensai naming; then git diff --exit-code in CI to prove freshness.
8. **Rebuild the binary.** bun build --compile; the version auto-restart writes the new git HEAD so the next command picks it up.
9. **Grep for stragglers.** Re-run the inventory; it must return zero for gstack/GStack/GSTACK/\$B (excluding CHANGELOG history and this migration doc).

9.3 What must NOT be renamed

- Upstream package imports (bun, Playwright, onnxruntime) — unrelated to naming.
- Historical CHANGELOG / commit messages — preserve provenance; note the rename at the top instead.
- Third-party model directories already downloaded — provide a migration shim rather than forcing a re-download.
- gbrain — it is a separate upstream project (garrytan/gbrain). Keep its name; only rename SenSai's *references* to how it's invoked.

9.4 New: the SENSAI.md schema (skeleton)

Replaces gstack's CLAUDE.md and adds the wiki operating rules. This is the single most important file — it makes the agent a disciplined wiki maintainer, not a generic chatbot.

```
# SENSAI.md — schema & operating manual

## Identity
You maintain the CEO's knowledge wiki and render the Heads-Up Display.
You do the bookkeeping; the CEO curates and decides.

## Layers
- raw/   : immutable. Read, never write.
- wiki/  : you own it. Compile, cross-link, keep current.

## Page types
client | person | decision | commitment | concept | hud   (see frontmatter contract)

## Operations
- INGEST: read source → discuss → update 10-15 pages → update index/overview → log.
- QUERY : read index first → drill in → answer with citations → file good answers back.
- LINT  : scoped to changed pages + graph neighbors; full sweep on request only.
- BRIEF : compile wiki/hud/today.md from active pages (see PRD §6).

## Gates
- Contradiction severity: none|soft|scope|hard. HARD blocks the ingest commit.
- Commit gate: grep wiki/ for 'Status: Unresolved' (no LLM) before any commit.
- Exec Assistant: NO external send / calendar write without CEO approval. Log it.

## Conventions
```

- Frontmatter required on every page. Cross-link with `[[wikilinks]]`.
- `log.md` entries: `'## [YYYY-MM-DD] ingest|query|lint|brief|assist | <title>'`.
- Never delete commitments; mark superseded.

10. Delivery plan & phasing

Phase	Scope	Exit criteria
P0 — Rename	Execute §9 migration; SENSAL.md skeleton; binary rebuilt	Zero gstack refs; all inherited tests green
P1 — Wiki core	raw/wiki/ layout, ingest+query+lint, index/log, contradiction gate	Single transcript → compiled client page with citations
P2 — Client Notes	Client page structure, commitments ledger, sentiment flags	Commitments surface with owner/due/status
P3 — HUD	/sensai brief renders today.md; sections & ranking	Morning brief with sourced lines, empty-state safe
P4 — Exec Assistant	Drafting, meeting prep, chasing, approval gate	Zero un-approved external sends; Handled section populated
P5 — Scale (opt.)	Hybrid search / gbrain graph, scoped lint automation	Sub-second query at hundreds of pages

10.1 Risks & mitigations

Risk	Mitigation
Rename breaks a generated doc path	Docs are generated; CI git-diff gate proves freshness before merge
Wiki drift / stale summaries	Lint is not optional; run on a timer; lifecycle status per page
Cross-client contamination in synthesis	Scope evidence to the client's page + linked neighbors; no cross-entity inference
Assistant sends something wrong	Hard approval gate + full audit log in v1; no autonomous outbound
Private client data leaks to cloud	Local-first default; explicit logged boundary for any cloud synthesis; redaction tags
Fork already diverged from upstream	Reconcile §9 against fork HEAD before executing the rename

11. Open questions for review

1. Should the HUD have a web view (stretch) or stay a Markdown surface in Claude Code / Cowork for v1?
2. Which connectors are must-have at P1 (email + calendar + transcripts) vs. deferred (CRM, Slack)?
3. Do we adopt gbrain's graph from P1 for people/client relationships, or stay index-first until scale forces it?
4. What is the exact sentiment-drop threshold that raises a client flag, and is it per-client tunable?
5. Approval UX: inline AskUserQuestion vs. a batched end-of-day approval queue for the assistant?

End of PRD — SenSai: CEO Heads-Up Display & Knowledge Wiki (Draft v1.0)